

Laporan Jurnal Praktikum

Pengembangan Aplikasi

Berbasis Web

**Modul 4: Mengoperasikan Database dan
Generate Data Otomatis dengan Laravel**



Nama : Ryandi Agung Mazhendra

Nim : 707012500024

Kelas: 49-04

D4SIKC

4.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep koneksi database pada Laravel.
2. Membuat koneksi database melalui file .env.
3. Membuat dan menjalankan migration untuk membentuk tabel database.
4. Membuat form input data mahasiswa.
5. Melihat dan memastikan data tersimpan ke dalam database.
6. Mampu mengimplementasikan database seeding untuk menghasilkan data secara otomatis dalam database.

4.2 Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

Perangkat Keras :

- Komputer atau laptop dengan sistem operasi Windows/Linux/MacOS.
- Minimal prosesor 1 GHz, RAM 4GB (disarankan 8GB), serta ruang penyimpanan kosong minimal 2GB.

Perangkat Lunak:

- XAMPP atau Laragon sebagai server lokal (Apache & MySQL).
- Composer sebagai dependency manager PHP.
- PHP versi 8.0 atau lebih tinggi (sudah termasuk dalam XAMPP/Laragon).
- Laravel Framework (minimal versi 8) yang sudah terinstal melalui Composer.
- Teks Editor / IDE seperti Visual Studio Code, Sublime Text, atau PHPStorm.
- Browser Web seperti Google Chrome, Safari, atau Mozilla Firefox untuk menjalankan aplikasi.

4.3 Langkah – Langkah Praktikum

1. Konfigurasi Database

Membuka XAMPP dan menjalankan Apache dan MySql ,Kemudian membuat project laravel yang berjudul LaporBanjir

Membuka Mysql dan membuat database baru untuk project Laporbanjir, buka file .env yang ada di folder project LaporBanjir kemudian ubah bagian konfigurasi database seperti berikut:

```
DB_CONNECTION=mysql
```

```
DB_HOST=127.0.0.1
```

```
DB_PORT=3306
```

```
DB_DATABASE=db_laporbanjir
```

```
DB_USERNAME=root
```

```
DB_PASSWORD=
```

Setelah konfigurasi selesai, jalankan perintah *php artisan config:clear* agar perubahan konfigurasi dapat terbaca oleh Laravel.

Membuat Migration

Buat migration untuk tabel laporan dengan menjalankan perintah berikut pada terminal:

php artisan make:migration create_laporan_table

Setelah file migration dibuat, buka file tersebut di folder database/migrations/, kemudian isi struktur tabel laporan sesuai dengan kode yang digunakan.

```
return new class extends Migration
{
    public function up(): void
    {
        Schema::create('laporan', function (Blueprint $table) {
            $table->id();
            $table->string('nama_pelapor');
            $table->string('lokasi');
            $table->integer('tinggi_genangan');
            $table->date('tanggal_kejadian');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('laporan');
    }
};
```

Pada tabel ini terdapat data nama pelapor, lokasi, tinggi genangan, dan tanggal kejadian. Setelah selesai, jalankan *php artisan migrate* untuk membuat tabel tersebut di database. Kemudian buka phpMyAdmin dan cek database db_laporbanjir. Jika berhasil, tabel laporan akan muncul di dalam database.

2. Membuat Model

Buat model Laporan menggunakan perintah: *php artisan make:model Laporan*

Setelah itu buka file app/Models/Laporan.php dan sesuaikan isinya dengan kode yang digunakan.

```

class Laporan extends Model
{
    protected $table = 'laporan';

    protected $fillable = [
        'nama_pelapor',
        'lokasi',
        'tinggi_genangan',
        'tanggal_kejadian',
    ];

    protected $casts = [
        'tanggal_kejadian' => 'date',
    ];

    public function getStatusAttribute()
    {
        if ($this->tinggi_genangan >= 100) {
            return 'Awas';
        } elseif ($this->tinggi_genangan >= 50) {
            return 'Siaga';
        }
        return 'Waspada';
    }

    public function getLebarBarAttribute()
    {
        return min(100, ($this->tinggi_genangan / 150) * 100);
    }
}

```

Bagian \$fillable digunakan untuk menentukan data yang boleh dimasukkan ke database. Pada model ini juga dibuat penentuan status berdasarkan tinggi genangan. Jika tinggi genangan 100 cm atau lebih maka statusnya Awas, 50 cm atau lebih menjadi Siaga, sedangkan di bawah 50 cm menjadi Waspada. Dengan cara ini status bisa langsung dipanggil melalui \$laporan->status.

3. Membuat Controller

Buat controller dengan perintah: *php artisan make:controller LaporanController* Controller ini digunakan untuk mengatur proses yang ada pada aplikasi. Di dalamnya terdapat fungsi untuk menampilkan beranda, menampilkan form, menyimpan laporan, dan menampilkan daftar laporan.

```

class LaporanController extends Controller
{
    public function beranda()
    {
        $semua = Laporan::all();

        return view('beranda', [
            'total' => $semua->count(),
            'awas' => $semua->where('status', 'Awas')->count(),
            'terbaru' => $semua->sortByDesc('tanggal_kejadian')->take(3),
        ]);
    }

    public function form()
    {
        return view('form');
    }

    public function simpan(Request $request)
    {
        $request->validate([
            'nama_pelapor' => 'required|string|max:255',
            'lokasi' => 'required|string|max:255',
            'tinggi_genangan' => 'required|integer|min:1|max:500',
            'tanggal_kejadian' => 'required|date|before_or_equal:today',
        ]);
    }
}

```

```

        $laporan = Laporan::create([
            'nama_pelapor' => $request->nama_pelapor,
            'lokasi' => $request->lokasi,
            'tinggi_genangan' => $request->tinggi_genangan,
            'tanggal_kejadian' => $request->tanggal_kejadian,
        ]);

        return view('konfirmasi', ['laporan' => $laporan]);
    }

    public function daftar()
    {
        $data = Laporan::orderByDesc('tanggal_kejadian')->get();

        return view('daftar', [
            'laporan' => $data,
            'jumlah' => [
                'semua' => $data->count(),
                'awas' => $data->where('status', 'Awas')->count(),
                'siaga' => $data->where('status', 'Siaga')->count(),
                'waspada' => $data->where('status', 'Waspada')->count(),
            ],
        ]);
    }
}

```

Pada fungsi simpan(), data dari form diperiksa terlebih dahulu menggunakan validasi. Jika data sudah sesuai, data tersebut disimpan ke database menggunakan model Laporan.

4. Routing

Buka file routes/web.php, kemudian tambahkan route yang digunakan pada aplikasi.

Route / digunakan untuk menampilkan halaman beranda, /form untuk menampilkan form laporan, /simpan untuk menerima data dari form, dan /daftar-laporan untuk menampilkan laporan yang sudah tersimpan.

```
Route::get('/', [LaporanController::class, 'beranda']);
Route::get('/form', [LaporanController::class, 'form']);
Route::post('/simpan', [LaporanController::class, 'simpan']);
Route::get('/daftar-laporan', [LaporanController::class, 'daftar']);
```

Route /simpan menggunakan method POST karena digunakan untuk mengirim data, sedangkan route lainnya menggunakan GET karena digunakan untuk menampilkan halaman.

5. Membuat View

Pada tahap ini dibuat beberapa file Blade yang digunakan untuk tampilan aplikasi. File tersebut terdiri dari komponen alert, layout utama, partial kartu laporan, halaman beranda, form laporan, halaman konfirmasi, dan halaman daftar laporan.

a. Komponen <x-alert>

```
@props(['type' => 'success'])

<div class="alert alert-{{ $type }}">
    {{ $slot }}
</div>
```

File components/alert.blade.php digunakan sebagai komponen untuk menampilkan pesan seperti berhasil atau terjadi kesalahan. Isi pesan dapat dimasukkan melalui \$slot.

b. Layout utama

File layouts/app.blade.php digunakan sebagai template utama untuk halaman aplikasi. Di dalamnya terdapat navbar, footer, CSS, dan @yield('content') yang nantinya diisi oleh halaman lain.

c. Partial kartu laporan

```
<div class="card status-{{ strtolower($laporan->status) }}">
    <div class="card-top">
        <span class="badge badge-{{ strtolower($laporan->status) }}">{{ $laporan->status }}</span>
        <span class="tgl">{{ $laporan->tanggal_kejadian->format('d M Y') }}</span>
    </div>
    <h3>📍 {{ $laporan->lokasi }}</h3>
    <p class="tinggi">{{ $laporan->tinggi_genangan }} <small>cm</small></p>
    <p class="pelapor">Dilaporkan oleh {{ $laporan->nama_pelapor }}</p>
</div>
```

File partials/laporan-card.blade.php digunakan untuk menampilkan data laporan dalam bentuk kartu. File ini dibuat sebagai partial supaya tampilan kartu laporan tidak perlu ditulis berulang kali di beberapa halaman.

d. Halaman beranda

File `beranda.blade.php` menampilkan halaman awal aplikasi. Pada halaman ini terdapat jumlah laporan, jumlah laporan dengan status *Awas*, dan beberapa laporan terbaru.

Data laporan ditampilkan menggunakan `@foreach`, sedangkan kartu laporan dipanggil menggunakan `@include`.

e. Halaman form

File `form.blade.php` digunakan untuk memasukkan data laporan banjir. Data yang diisi meliputi nama pelapor, lokasi, tinggi genangan, dan tanggal kejadian.

Form menggunakan method `POST` dan `@csrf` agar data dapat dikirim ke route `/simpan`.

f. Halaman konfirmasi

File `konfirmasi.blade.php` digunakan untuk menampilkan pemberitahuan setelah laporan berhasil disimpan. Data laporan yang baru dimasukkan juga ditampilkan kembali menggunakan partial kartu laporan.

g. Halaman daftar laporan

File `daftar.blade.php` digunakan untuk menampilkan seluruh laporan yang sudah tersimpan di database. Pada halaman ini juga ditampilkan jumlah laporan berdasarkan status *Waspada*, *Siaga*, dan *Awas*.

6. Membuat Seeder

Buat seeder menggunakan perintah: `php artisan make:seeder LaporanSeeder`

Kemudian buka file `database/seeder/LaporanSeeder.php` dan masukkan kode seeder yang sudah dibuat.

```
class LaporanSeeder extends Seeder
{
    public function run(): void
    {
        $faker = Faker::create('id_ID');

        $kecamatan = ['Dayeuhkolot', 'Baleendah', 'Bojongsoang', 'Rancaekek', 'Majalaya'];

        for ($i = 0; $i < 20; $i++) {
            DB::table('laporan')->insert([
                'nama_pelapor' => $faker->name,
                'lokasi' => $faker->randomElement($kecamatan),
                'tinggi_genangan' => $faker->numberBetween(10, 150),
                'tanggal_kejadian' => $faker->dateTimeBetween('-1 month', 'now')->format('Y-m-d'),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }
    }
}
```

Seeder ini digunakan untuk membuat data laporan secara otomatis tanpa harus mengisi form satu per satu. *Faker* digunakan untuk membuat nama, lokasi, tinggi genangan, dan tanggal secara acak. Pada bagian kecamatan digunakan beberapa pilihan lokasi yang sudah ditentukan. Tinggi genangan dibuat secara acak dari 10 sampai 150 cm sehingga data yang dihasilkan bisa memiliki status yang berbeda-beda. Kemudian tambahkan `LaporanSeeder` ke dalam `$this->call()`.

```

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            LaporanSeeder::class,
        ]);
    }
}

```

Dengan begitu, LaporanSeeder dapat dijalankan melalui DatabaseSeeder ketika proses seeding dilakukan.

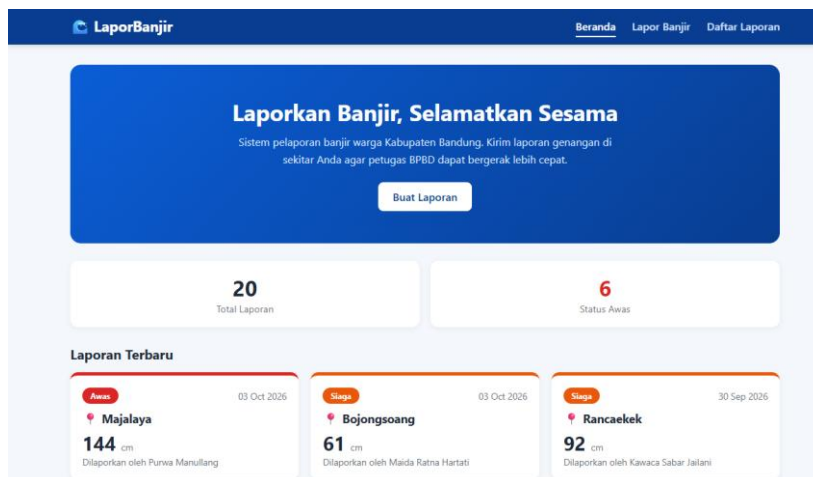
Untuk menjalankan seeder, gunakan perintah: `php artisan db:seed --class=LaporanSeeder`

Setelah dijalankan, data laporan akan otomatis masuk ke tabel laporan. Jika ingin menghapus data lama sekaligus membuat ulang tabel dan menjalankan semua seeder, dapat menggunakan: `php artisan migrate:fresh --seed`

4.4 Dokumentasi Program dan hasil

Web LaporBanjir

1. Beranda



2. Form LaporBanjir

3. Daftar Laporan

TANGGAL	LOKASI	TINGGI GENANGAN	STATUS	PELAPOR
03 Oct 2026	Majalaya	144 cm	Awat	Purwa Manullang
03 Oct 2026	Bojongsong	61 cm	Siaga	Maida Ratna Hartati
30 Sep 2026	Rancaekek	92 cm	Siaga	Kawaca Sabar Jailani
29 Sep 2026	Baleendah	57 cm	Siaga	Faizah Pudjiastuti
28 Sep 2026	Bojongsong	110 cm	Awat	Cecep Joko Lazuardi
28 Sep 2026	Bojongsong	69 cm	Siaga	Bella Yuniar

4. Data database

id	nama_pelapor	lokasi	tinggi_genangan	tanggal_kejadian	created_at	updated_at
1	Purwa Fingantoro	Bojongsong	108	2026-09-15	2026-10-06 11:55:59	2026-10-06 11:55:59
2	Karsana Rudi Kusumo S.H	Baleendah	93	2026-09-21	2026-10-06 11:55:59	2026-10-06 11:55:59
3	Purwa Manullang	Majalaya	144	2026-10-03	2026-10-06 11:55:59	2026-10-06 11:55:59
4	Pandi Siregar S.Kel	Majalaya	134	2026-09-25	2026-10-06 11:55:59	2026-10-06 11:55:59
5	Gawati Pratiwi	Majalaya	24	2026-09-07	2026-10-06 11:55:59	2026-10-06 11:55:59
6	Maik Suryono	Bojongsong	26	2026-09-21	2026-10-06 11:55:59	2026-10-06 11:55:59
7	Bahuwirya Mustofa	Dayeuhkolot	93	2026-09-21	2026-10-06 11:55:59	2026-10-06 11:55:59
8	Maida Ratna Hartati	Bojongsong	61	2026-10-03	2026-10-06 11:55:59	2026-10-06 11:55:59
9	Wardaya Sitorang	Bojongsong	45	2026-09-21	2026-10-06 11:55:59	2026-10-06 11:55:59
10	Nurul Makha Nurani S.H	Majalaya	80	2026-09-18	2026-10-06 11:55:59	2026-10-06 11:55:59
11	Vivi Purnawati	Baleendah	136	2026-09-15	2026-10-06 11:55:59	2026-10-06 11:55:59
12	Faizah Pudjiastuti	Baleendah	57	2026-09-29	2026-10-06 11:55:59	2026-10-06 11:55:59
13	Cecep Joko Lazuardi	Bojongsong	110	2026-09-28	2026-10-06 11:55:59	2026-10-06 11:55:59
14	Bella Yuniar	Bojongsong	69	2026-09-28	2026-10-06 11:55:59	2026-10-06 11:55:59
15	Raden Maulana M.Pd	Majalaya	92	2026-09-14	2026-10-06 11:55:59	2026-10-06 11:55:59
16	Harsana Kusarjono S.KP	Majalaya	27	2026-09-16	2026-10-06 11:55:59	2026-10-06 11:55:59
17	Samsul Harsanto Situmorang	Baleendah	85	2026-09-14	2026-10-06 11:55:59	2026-10-06 11:55:59
18	Syahrini Pompa	Dayeuhkolot	34	2026-08-22	2026-10-06 11:55:59	2026-10-06 11:55:59
19	Cahyanto Sinega	Majalaya	127	2026-09-24	2026-10-06 11:55:59	2026-10-06 11:55:59
20	Kawaca Sabar Jailani	Rancaekek	92	2026-09-30	2026-10-06 11:55:59	2026-10-06 11:55:59

4.5 Pembahasan dan Analisis

Analisis. Aplikasi LapoBanjir berjalan dengan alur form → route → controller → model → database. Pengguna mengisi data pada /form, kemudian data dikirim menggunakan method POST ke route /simpan dan diteruskan ke method simpan() pada LaporanController. Data yang sudah lolos validasi kemudian disimpan ke tabel laporan menggunakan model Laporan. Saat halaman /daftar-laporan dibuka, data laporan diambil dari database dan ditampilkan dalam bentuk kartu berdasarkan tanggal kejadian. Status laporan juga ditentukan dari tinggi genangan, yaitu Waspada untuk genangan di bawah 50 cm, Siaga mulai 50 cm, dan Awat mulai 100 cm. Selama pengerjaan, beberapa kendala yang dapat muncul antara lain database tidak terhubung karena konfigurasi .env belum sesuai, tabel belum terbentuk karena migration belum dijalankan, error saat mengirim form karena @csrf belum ditambahkan, serta data tidak dapat disimpan jika nama kolom pada model dan migration tidak sesuai.

Pembahasan. Migration digunakan untuk membuat tabel laporan yang berisi data nama pelapor, lokasi, tinggi genangan, dan tanggal kejadian. Model Laporan digunakan untuk menghubungkan aplikasi dengan tabel tersebut dan mengatur data yang boleh disimpan melalui \$fillable. Eloquent juga membuat proses menyimpan dan mengambil data menjadi lebih sederhana karena tidak perlu menulis query SQL secara langsung. Selain itu, penggunaan accessor pada model membuat status genangan dapat langsung dipanggil menggunakan \$laporan->status, sehingga proses penentuan status tidak perlu ditulis ulang pada setiap view. Seeder dan Faker digunakan untuk membuat data laporan secara otomatis sehingga pengujian halaman daftar laporan dapat dilakukan dengan banyak data tanpa harus mengisi form satu per satu. Penggunaan migrate:fresh --seed juga mempermudah saat ingin mengosongkan database dan membuat kembali tabel beserta data seeder pada tahap pengembangan.

4.6 Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa Laravel dapat digunakan untuk membuat aplikasi pelaporan banjir yang terhubung dengan database MySQL. Data laporan yang dimasukkan melalui form dapat divalidasi, disimpan ke database, dan ditampilkan kembali pada halaman daftar laporan. Penggunaan migration dan model memudahkan pengelolaan data, sedangkan accessor digunakan untuk menentukan status genangan berdasarkan tinggi air. Selain itu, seeder dan Faker membantu membuat data laporan secara otomatis sehingga pengujian aplikasi menjadi lebih mudah. Dengan praktikum ini, saya dapat memahami cara menghubungkan form, route, controller, model, dan database dalam satu aplikasi web.

4.7 Link Github

<https://github.com/agungmryandi-stack/PABW>